keys is `O(1)`, but rather `O(m)`, depending on the length `m` of the shortest of the two keys.

Table 6.2 Running time for operations on tries vs balanced BSTs, assuming `n` keys with average length `M`; finally, as a simplification we assume for the size of the input keys `m`, that $m \in O(M)$.

Method	BST	BST + hash	Trie
`Search`	O(m*log(n))	O(m+log(n))	O(m)
`insert`	O(m*log(n))	O(m+log(n))	O(m)
`remove`	O(m*log(n))	O(m+log(n))	O(m)
`longestPrefix`	O(m*n)	O(m+n)	O(m)
`keysWithPrefix`	O(m*n)	O(m+n)	O(n+m) §

(§) average

The third column in table 6.2 shows the results for a particular variant of BSTs, where we store a hash of the string, together with the key itself, in each node. This approach, that requires an extra `O(n)` memory to store these fields, allows for a fast two-pass comparison: given a search string `w`, we compute `h(w)` before starting the search; then for each node we first check if `h(w)` matches the node's hash (which requires constant time), and only when it does, we perform a proper strings comparison.